

```

    last(d. peekFirst());
    last(d. peekLast());
    last(d);
}

```

Day 2 dry run

① Doubly Linked List

```

node
{
    prev
    next
}

```

1) addLast(20)	20
2) addLast(40)	20 $\rightleftharpoons$ 40
3) addFirst(10)	10 $\rightleftharpoons$ 20 $\rightleftharpoons$ 40
4) add(2, 30)	10 $\rightleftharpoons$ 20 $\rightleftharpoons$ 30 $\rightleftharpoons$ 40

add(2, 30) = nodeAt(2), size = 3 & 2 $\neq$ & is not less than 3/2 = 1, so it starts from tail.

- tail 40 at index 2

n.prev $\rightleftharpoons$ 20, n.next $\rightleftharpoons$ 40, 20.next $\rightleftharpoons$ n, 40.prev $\rightleftharpoons$ n

→ removeLast $\rightleftharpoons$ 40 and removeFirst $\rightleftharpoons$ 10
leaving 20 $\rightleftharpoons$ 30

→ ~~to remove(20)~~

remove(10) = 30

1000 nodes

get(10) = 10 steps from head

get(990) = 9 steps from tail

get(500) = 499 steps from tail

10 $\Rightarrow$ 20 $\Rightarrow$ 30 $\Rightarrow$ 40

40 removed

10 removed

20 removed

final = 30

② Josephus Problem

$n=7$ $k=3$ start : 1 2 3 4 5 6 7

every 3rd person is removed

Round 1

1 2 4 5 6 7

Round 2

1 2 4 5 7

Round 3

1 4 5 7

Round 4

1 4 5

Round 5Round 6Survivor = 4

for $n = 41$, $k = 3$
 Survivor = 31

③ Linked Circular Queue

Capacity = 4

enqueue(10) $\rightarrow$ 10enqueue(20) $\rightarrow$ 10 20enqueue(30) $\rightarrow$ 10 20 30

dequeue() - 20 30

enqueue(40) - 20 30 40

enqueue(50) - 20 30 40 50

enqueue(60) - queue overflow

dequeue $20 \rightarrow 30 \rightarrow 40 \rightarrow 50$ ④ Linked Queue

enqueue(10) = 10

enqueue(20) = 10 20

enqueue(30) = 10 20 30

front = 10

rear = 30

dequeue

dequeue() → 10

Queue () → 20 30

dequeue() → 20

dequeue() → 30

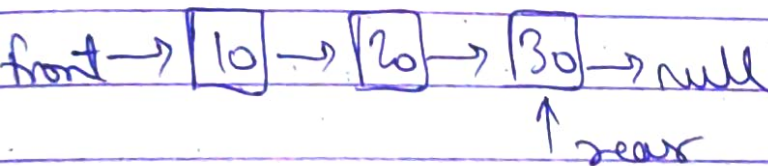
Queue = empty empty

front = null

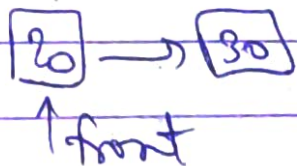
rear = null

then, enqueue(40)

queue = 40, front = 40, rear = 40



dequeue



⑤ Linked Stack

push(A) → A

push(B) → B → A

push(C) $\rightarrow C \rightarrow B \rightarrow A$

pop() = C Stack $\rightarrow B \rightarrow A$ peek() = B

Top
↓
C
↓
B
↓
A
↓
null

push D

Top
↓
D
↓
C
↓
B
↓
A

pop D

Top
↓
C
↓
B
↓
A

⑧ List Iterator Demo

remove even number

- 1) linkedlist + Iterator.remove() = $O(n)$
- 2) ArrayList + Iterator.remove() = $O(n^2)$
- 3) ArrayList + removeIf() $\Rightarrow O(n)$

1) 10 - remove
15 keep
20 remove
25 keep
final = 15 25

2) 10 15 20 25
10 remove 15 20 25
20 remove 15 25

3) 10 15 20 25 30
if (num % 2 == 0)
then $O(n)$

$n = 50,000$

$n = 100,000$

$n = 200,000$

n doubles : 50,000 $\rightarrow$ 100,000 $\rightarrow$ 200,000

$$O(n) = 2x$$

$$O(n^2) = 4x$$

7. LRU Cache

capacity = 3

put(home)

put(about)

put(login)

[home, about, login]

get(home): [about, login, home]

put(cart): capacity 3 so about removed
[login, home, cart]

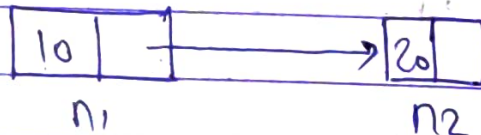
8. Temp. java

$$n_1 = 10$$

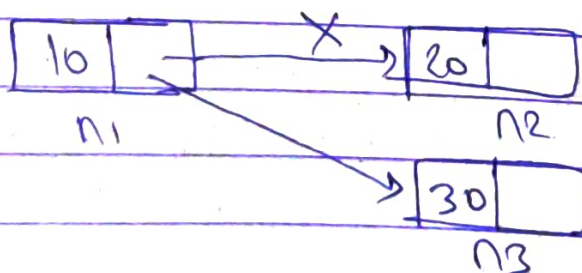
$$n_2 = 20$$

$$n_3 = 30$$

$$n_1.\text{next} = n_2$$



$$n_1.\text{next} = n_3$$



display output 10 30

output 10 20 10 30

③ ArrayLinkedList

capacity = 6

head = -1

free = 0

addFirst(30)

data[0] = 30

head = 0

free = 1

addFirst(10)

data[1] = 10

next[1] = 0

head = 1

list 10 → 30

addAfter(10, 20)

data[2] = 20

10 → 20 → 30

remove(10), head = 2

20 → 30

addFirst(5)

data[1] = 5

head = 1

5 → 20 → 30 → null

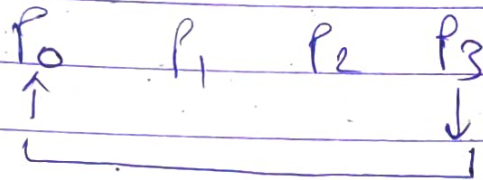
⑩ Circular Linked List

addLast(P_1) → P_1

addLast(P_2) → $P_1 P_2$

addLast(P_3) → $P_1 P_2 P_3$

addFirst(P_0) → $P_0 P_1 P_2 P_3$



Round Robin

7 processes

Slave 1 → P_0

Slave 2 → P_1

Slave 3 → P_2

Slave 4 → P_3

Slave 5 → P_0

Slave 6 → P_1

Slave 7 → P_2

removeFirst() = P_3

$P_0 \rightarrow P_1 \rightarrow P_2 \rightarrow$ back to P_0

Linked ① Singly List

head $\rightarrow$ null

insertAtBeginning(10)

newNode = [10 | null]

head = newNode

head $\rightarrow$ [10] $\rightarrow$ null

insertAtBeginning(20)

20.next = old head 10

head $\rightarrow$ [20] $\rightarrow$ [10] $\rightarrow$ null

insertAtBeginning(30)

head $\rightarrow$ [30] $\rightarrow$ [20] $\rightarrow$ [10] $\rightarrow$ null

insertAtEnd(40)

10.next = 40

head $\rightarrow$ [30] $\rightarrow$ [20] $\rightarrow$ [10] $\rightarrow$ [40] $\rightarrow$ null

search(10)

30 $\rightarrow$ no

20 $\rightarrow$ no

10 $\rightarrow$ yes

Result = true

deleteAtBeginning()

30 remove:

before head $\rightarrow$ 30 $\rightarrow$ 20 $\rightarrow$ 10 $\rightarrow$ 40

after head $\rightarrow$ 20 $\rightarrow$ 10 $\rightarrow$ 40 $\rightarrow$ null

insertAtPosition (25, 2)

Before $20 \rightarrow 10 \rightarrow 40$

After $20 \rightarrow 25 \rightarrow 10 \rightarrow 40$

deleteAtEnd $20 \rightarrow 25 \rightarrow 10 \rightarrow \text{null}$

~~deleteAtEnd~~

deleteByValue (25) $20 \rightarrow 10 \rightarrow \text{null}$

reverse()

↳ $\begin{matrix} 20 & 10 & \text{null} \\ 10 & 20 & \text{null} \end{matrix}$

Final

head $\rightarrow [10] \rightarrow [20] \rightarrow \text{null}$

② ⑫ Doubly Linked List

[pre | data | next]

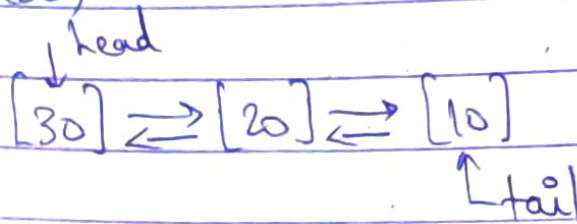
addFirst (10)

head / tail
↓
[null | 10 | null]

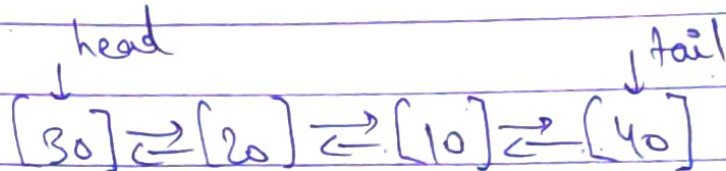
addFirst (20)

head
↓
[null | 20 | next] $\Rightarrow$ [prev | 10 | null]
↑
tail

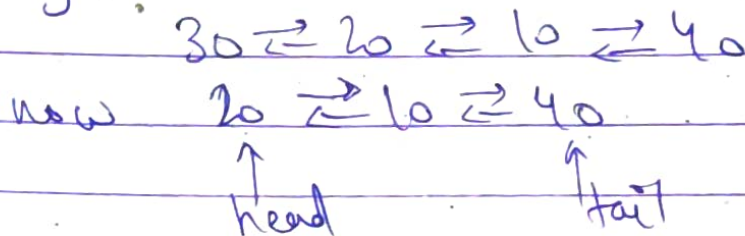
addFirst(30)



addLast(40)



deleteAtBeginning()



deleteAtEnd()

~~40~~ 20 10 40



deleteByValue(10)

before = 20 -> 10 -> 40
after = 20 -> 40

add(1, 30)

before 20 -> 40
after 20 -> 30 -> 40

③ Demo linked list Demo

insertAtBeginning(10) $\rightarrow$ 10

insertAtEnd(40) $\rightarrow$ 10 40

display - 10 40

insert

search(10) 10 $\rightarrow$ yes
result true

take another

list - 10 20 30 40

search(99)

10 $\rightarrow$ no

20 $\rightarrow$ no

30 $\rightarrow$ no

40 $\rightarrow$ no

null $\rightarrow$ false

deleteAtBeginning():

Before 30 10 $\rightarrow$ 20 $\rightarrow$ 30 $\rightarrow$ 40
after 20 $\rightarrow$ 30 $\rightarrow$ 40